Robotics French Cup Strategy Simulator

Agent Author's Guide

Contents

1 Overview

A 2-player game simulating the 2026 Robotics French Cup, to be played by python agents. Two circular robots compete on a rectangular map to collect, recolor, and deposit "boxes" for points.

As an agent author, you will:

- write Python modules in `agents/` that decide what your robot does each tick,
- select which agent plays which player in `agents_config.py`,
- run and watch games with the `play.py` launcher.

2 The game, conceptually

The map
A rectangle containing non-overlapping rectangular areas: player 0's area, player 1's area, and one or more *scoring areas*.

Boxes
Identical-sized rectangles scattered on the map. Each box has a **color** (`0` or `1`) and can be picked up by robots.

Robots
Each player controls one circular robot starting in their own area. A robot can rotate freely, move forward, pick up a nearby box, change the color of a nearby box, or lay down a box it is holding. Moving, picking up, recoloring and laying down each take real time: they put the robot on a temporary *cooldown* during which it cannot act at all (see Section 3).

Turns
Time is discretized into *ticks*. Each tick, player 0 then player 1 gets to call `make_decision()` once. **Exactly one action** may be taken per turn.

Scoring
Computed at game end:

- player X wins +`POINTS_OWN_AREA` points for every box sitting in X's own area (regardless of the box's color);
- player X wins +`POINTS_OWN_AREA` points for every box sitting in a scoring area whose color is X;
- boxes currently held by a robot score nothing.

3 Game rules in detail

- **One action per turn**: `make_decision()` may call exactly one action function. `rotate` doesn't count as an action and can be called an arbitrary number of times every turn (subject to the cooldown rule below).

- **Lay-down placement**: fixed distance (`LAY_DOWN_DISTANCE`) directly in front of the robot, oriented along the robot's current facing direction. Fails if the spot is out of map bounds or overlaps another box.

- **Box rotation**: boxes take the robot's orientation when laid down.

- **Initial layout**: positions, orientations and areas are fully config-defined and fixed. Box colors are also config-defined by default, unless the game is started with `python play.py run -random-colors`, which assigns each box a random color (0 or 1) instead.

- **Failed actions don't consume the turn**: if an action raises `GameError` (e.g. `lay_down` blocked, `pickup` out of reach), your turn is *not* used up, so you can try something else.
- **Cooldown / temporal cost**: `move`, `pickup`, `set_color` and `lay_down` each occupy the robot for a number of ticks given by their own constant — `MOVE_COOLDOWN`, `PICKUP_COOLDOWN`, `SET_COLOR_COOLDOWN` and `LAY_DOWN_COOLDOWN` respectively. A value of N means the robot is occupied for N *total* ticks, including the tick the action succeeds on (so $N = 1$ means no extra delay, and $N > 1$ means $N - 1$ additional frozen ticks). While occupied, **any** action function — including `rotate` — raises `GameError` without consuming the turn. `rotate` itself never starts a cooldown. Call `get_cooldown(player)` to check how many ticks remain (0 = free).
- **Movement collision**: the robot stops just before hitting the map edge, the other robot, or a laid-down box.
- **Held boxes don't score** and are not physical obstacles for movement.

4 Getting started

4.1 Running a game

Use `play.py` with your regular system Python (`python` / `python3` / `py`) from the project root.

```
python play.py run                      # run a game, print progress +
    final score
python play.py run --quiet              # suppress per-tick error
    logging
python play.py run --save               # also save a timestamped game
    to saved_games/
python play.py run --save --view        # run, save, then immediately
    replay it
python play.py run --random-colors      # randomize each box's starting
    color
python play.py view <history.json>      # replay a game from saved_games
    / (or any path)
```

4.2 Choosing which agent plays

Edit `agents_config.py` at the project root:

```
PLAYER0_AGENT = "agents.simple_agent"
PLAYER1_AGENT = "agents.idle_agent"
```

Each string is a module path under `agents/`, loaded dynamically. Both slots can name the *same* module to make an agent play against itself.

5 Writing your own agent

Create `agents/my_agent.py` defining a class `Agent` with a `make_decision(self)` method. Don't forget to `import interface` and `import config`.

```
import interface
import config

class Agent:
    def make_decision(self):
```

```
        player = interface.me()
        accessible = interface.get_accessible_boxes(player)
        if accessible:
            interface.pickup(accessible[0].id)
        else:
            interface.move(10.0)
```

Then point `agents_config.PLAYER0_AGENT` (or `PLAYER1_AGENT`) at `"agents.my_agent"` and run `python play.py run -view`.

Look at `agents/random_agent.py` for an example.

5.1 Agent memory

The runner creates one `Agent()` *instance* per player, even when `PLAYER0_AGENT` and `PLAYER1_AGENT` both name the same module. That means your agent can have a memory if you need it:

```
class Agent:
    def __init__(self):
        self.visited = set()

    def make_decision(self):
        player = interface.me()
        self.visited.add(interface.get_position(player))
        # ...
```

6 The agent interface (`interface` module)

6.1 Information functions

Free to call, any number of times.

Function	Returns
`me()`	This agent's player index (0 or 1)
`opponent()`	`1 - me()`
`get_position(player)`	`(x, y)` center of player's robot
`get_orientation(player)`	Unit vector player's robot is facing
`get_map()`	The `Map` object (`.width`, `.height`, `.areas`)
`get_area_containing(position)`	The `Area` that contains `position`, or `None` if it is in no area
`get_area(area_id)`	The `Area` with that id, or `None`
`get_area_center(area_id)`	`(x, y)` center of that area
`get_boxes()`	All `Box` objects (on ground and held)
`get_box(box_id)`	The `Box` with that id, or `None`
`get_accessible_boxes(player)`	Boxes on the ground within reach of `player`
`get_boxes_held(player)`	Boxes currently held by `player`
`is_accessible(player, box_id)`	Is the box on the ground and within reach?
`is_colliding(player, amount)`	Would moving forward by `amount` collide (edge/robot/box)?
`get_obstacle_distance(player)`	Distance from `player`'s robot to the nearest obstacle (map edge, other robot, or laid-down box) along its current orientation; **not** capped at `MAX_MOVE_SPEED`, may be `inf`
`get_box_distance(player, box_id)`	Circle-to-rectangle distance, robot → box
`get_box_direction(player, box_id)`	Unit vector from robot center toward box center
`get_area_distance(player, area_id)`	Distance from robot center to the area rectangle (0 if inside)
`get_area_direction(player, area_id)`	Unit vector from robot center toward area center
`get_score(player)`	Score `player` would get if the game ended right now
`get_cooldown(player)`	Ticks remaining until `player`'s robot can act again (0 = free now)

6.2 Action functions

Exactly one per `make_decision()` (except `rotate`, which doesn't count as an action). All five, including `rotate`, are blocked while the robot is on cooldown — see Section 3.

Function	Effect
`move(amount)`	Move forward by `amount` (≥ 0; capped at `MAX_MOVE_SPEED`); stops just before any collision. Starts a cooldown of `MOVE_COOLDOWN` ticks
`rotate(new_direction)`	Face `new_direction`. Does not itself start a cooldown
`pickup(box_id)`	Pick up an accessible box (fails if too far or hands full). Starts a cooldown of `PICKUP_COOLDOWN` ticks

Function	Effect
`set_color(box_id, color)`	Recolor an accessible box to 0 or 1. Starts a cooldown of `SET_COLOR_COOLDOWN` ticks
`lay_down(box_id)`	Place a held box in front of the robot (fails if blocked / out of bounds). Starts a cooldown of `LAY_DOWN_COOLDOWN` ticks

All action functions may raise:

- `interface.ActionError` if a second action is attempted in the same turn.
- `game.GameError` if the action itself is invalid (out of reach, blocked, negative `move` amount, etc.) **or if the robot is still on cooldown from a previous action** (this applies to `rotate` too). This does **not** consume the turn, so you can recover and try something else.

7 Data classes

These are the objects returned by `interface` functions.

`Box(id, position, orientation, color, owner)`
A box on the map. `owner == -1` means it is sitting on the ground; otherwise `owner` is the index of the player currently holding it. `color` is `0` or `1`; `orientation` is a unit vector along its width axis.

`Area(id, type, x, y, width, height)`
An axis-aligned rectangular zone. `type` is `0` (player 0's area), `1` (player 1's area), or `2` (a scoring area).

`Map(width, height, areas)`
The static map: overall dimensions plus the list of all `Area` objects (player areas and scoring areas together).

8 Constants reference (`config` module)

Map

Constant	Value	Meaning
`MAP_WIDTH`	1000.0	Width of the playing field, in game units (the `x` extent; recall `(0, 0)` is the top-left corner).
`MAP_HEIGHT`	600.0	Height of the playing field (the `y` extent).

Areas

Constant	Value	Meaning
`PLAYER0_AREA`	(0, 200, 150, 200)	`(x, y, width, height)` rectangle defining player 0's home area.
`PLAYER1_AREA`	(850, 200, 150, 200)	Same, for player 1 (`Area.type == 1`).
`SCORING_AREAS`	list of 2 rects	List of `(x, y, width, height)` rectangles, each becoming an `Area` with `type == 2`.

Robot starting positions and orientations

Constant	Value	Meaning
`PLAYER0_START`	(75, 300)	`(x, y)` spawn position of player 0's robot — inside `PLAYER0_AREA`.
`PLAYER0_START_ANGLE`	0.0	Initial facing angle of player 0's robot, in **degrees** measured from the $+x$ axis ($0° =$ facing right, toward the center of the map).
`PLAYER1_START`	(925, 300)	Spawn position of player 1's robot.
`PLAYER1_START_ANGLE`	180.0	Initial facing angle of player 1's

Robot physics

Constant	Value	Meaning
`ROBOT_RADIUS`	20.0	Radius of the circular robot body.
`MAX_MOVE_SPEED`	10.0	The largest `|amount|` a single `move` call will actually apply, per tick.
`MAX_BOXES_HELD`	3	Maximum number of boxes a robot can carry at once.

Box dimensions

Constant	Value	Meaning
`BOX_WIDTH`	40.0	Width of a box.
`BOX_HEIGHT`	25.0	Height of a box.

Interaction distances

Constant	Value	Meaning
`ACCESSIBILITY_RADIUS`	25.0	Maximum circle-to-rectangle distance (robot center → nearest point on the box) at which a box counts as "accessible" (i.e. within reach for `pickup` and `set_color`).
`LAY_DOWN_DISTANCE`	45.0	Fixed distance from the robot's center to the *center* of a box placed via `lay_down`.

Scoring

Constant	Value	Meaning
`POINTS_OWN_AREA`	3	Points awarded to player X for each box sitting in X's own area.
`POINTS_SCORING_AREA`	5	Points awarded to player X for each box sitting in a scoring area **whose color matches** X.

Timing

Constant	Value	Meaning
`DT`	0.1	Seconds represented by one tick.
`GAME_DURATION`	60.0	Total game length, in seconds (= `DT` × `TOTAL_TICKS`).
`TOTAL_TICKS`	600	Number of ticks in a game.

Action costs (temporal)

Constant	Value	Meaning
`MOVE_COOLDOWN`	1	Total ticks `move` occupies the robot for, including the tick it succeeds on.
`PICKUP_COOLDOWN`	20	Total ticks `pickup` occupies the robot for, including the tick it succeeds on.
`SET_COLOR_COOLDOWN`	10	Total ticks `set_color` occupies the robot for, including the tick it succeeds on.
`LAY_DOWN_COOLDOWN`	20	Total ticks `lay_down` occupies the robot for, including the tick it succeeds on.

While any of these cooldowns is active (`get_cooldown(player) > 0`), **all** action functions — including `rotate` — raise `GameError` without consuming the turn.

Initial boxes

Constant	Value	Meaning
`INITIAL_BOXES`	list of 12 tuples	The fixed starting layout: each entry is `(x, y, angle_degrees, color)` — the box's initial center position, its orientation expressed as an angle in degrees from the $+x$ axis, and its starting `color` (`0` or `1`).